FURG - C3
Algoritmos para Engenharia de Automação

LISTA SOBRE PROGRAMAÇÃO COM STRUCTS EM C

Prática 1: Escreva um programa que define o tipo **Produto** como uma *struct* de C (usando a declaração **typedef**). Este tipo contém os membros **quantidade** (valor inteiro), **custo** (valor real) e **nome** (vetor de 20 caracteres). Crie dois produtos: **a = {120, 0.10, "porca"}** e **b = {80, 0.20, "parafuso"}**. Então imprima seus valores, um cada linha e com as colunas alinhadas.

Resultado esperado:
```
120  0.10  porca
 80  0.20  parafuso
```

Prática 2: Com base na atividade anterior, adicione um procedimento **imprime_produto()** que recebe como parâmetro um **Produto p**, e então imprime seus valores, com o mesmo formato de antes. Altere o programa **main** para apenas definir **a** e **b**, chamando então o procedimento criado.

Resultado esperado:
```
120  0.10  porca
 80  0.20  parafuso
```

Prática 3: Com base na atividade anterior, adicione uma função **valor_produto()** que recebe como parâmetro um **Produto p** e retorna um valor **float**, calculado como a quantidade multiplicada pelo custo de **p**. No programa **main**, depois de imprimir os mesmos produtos **a** e **b**, calcule seus valores individuais chamando a função criada e então imprima o valor total.

Resultado esperado:
```
120  0.10  porca
 80  0.20  parafuso
valor total: 28.00
```

Prática 4: Com base na atividade anterior, no lugar das variáveis individuais **a** e **b** crie o seguinte vetor de produtos: **Produto vp[2] = { {120, 0.10, "porca"}, {80, 0.20, "parafuso"} };** Ajuste o código para cada produto seja impresso separadamente e que o valor total seja calculado e impresso como antes (agora apenas referenciando os produtos dentro do vetor **vp**).

Resultado esperado:
```
120  0.10  porca
 80  0.20  parafuso
valor total: 28.00
```

Prática 5: Escreva um programa que define o tipo **Personagem** como uma *struct* de C. Este tipo contém os membros **nome** (10 caracteres), **poder** (inteiro), **defesa** (inteiro) e **vida** (inteiro). Crie dois personagens: ("Xil", 54, 12, 100) e ("Yen", 40, 20, 100). Crie também um procedimento **status()**, que recebe um personagem como parâmetro e imprime seu estado atual. Chame este procedimento na função **main** para mostrar o *status* dos dois personagens.

Resultado esperado:

```
Xil poder=054 defesa=012 vida=100
Yen poder=040 defesa=020 vida=100
```

Prática 6: Com base na atividade anterior, adicione uma função **ataque()**, que recebe como parâmetro dois personagens **p1** e **p2**. Essa função calcula o dano causado pelo ataque de **p1** sobre **p2**, usando a fórmula **dano = p1.poder - p2.defesa** (se o cálculo for negativo, o dano deve ser zero). O resultado inteiro do dano deve ser retornado. Na função **main**, calcule e mostre o dano que seria causado por um ataque de Xil a Yen, e depois o dano que seria causado pelo ataque de Yen a Xil.

Resultado esperado:

```
Xil poder=054 defesa=012 vida=100
Yen poder=040 defesa=020 vida=100
ataque de Xil a Yen vai causar 34 de dano
ataque de Yen a Xil vai causar 28 de dano
```

Prática 7: Com base na atividade anterior, altere o programa principal para fazer uma batalha entre os dois personagens. Faça uma repetição enquanto os dois personagens estiverem com vida maior que zero. Dentro da repetição, calcule o dano do ataque de Xil a Yen, e depois de Yen a Xil. Só então atualize a vida dos personagens subtraindo o dano recebido. Finalmente, imprima o *status* de ambos e repita. Ao final, indique quem venceu ou se ambos morreram.

Resultado esperado:

```
Batalha:
      Xil poder=054 defesa=012 vida=072
      Yen poder=040 defesa=020 vida=066
      Xil poder=054 defesa=012 vida=044
      Yen poder=040 defesa=020 vida=032
      Xil poder=054 defesa=012 vida=016
      Yen poder=040 defesa=020 vida=-02
Xil venceu
```

Exercício 8: Com base na atividade anterior, adicione uma função **inimigo()**, que não recebe parâmetros. Essa função deve construir um novo personagem da raça "Zed", com poder, defesa e vida zero. Então use a função **rand()** para gerar valores aleatórios inteiros para o poder no intervalo [20, 30], para a defesa no intervalo [10, 30] e para a vida no intervalo [20, 50], devolvendo o personagem completo como valor de retorno. Faça então a batalha de Xil contra este inimigo gerado randomicamente.

Exercício 9: Com base na atividade anterior, crie um grupo com cinco inimigos gerados aleatoriamente (guardados em um vetor de **Personagem**). Faça então a batalha transcorrer em *rounds*, onde o herói Xil luta contra cada inimigo ainda vivo de cada vez.